Compiler Construction Lab - Assignment 10

Name: Dev Vachhani
PRN.: 24070122517
Batch: B3

Objective

To design and implement a parser that checks whether an input string can be generated by a user-defined context-free grammar. The program accepts production rules with alternatives and eps productions, displays the parsing steps in a table, and reports whether the string is accepted or rejected.

Flex tokenizes the grammar notation, and Bison parses the production lines. The input string is parsed by hand-written recursive-descent functions in C.

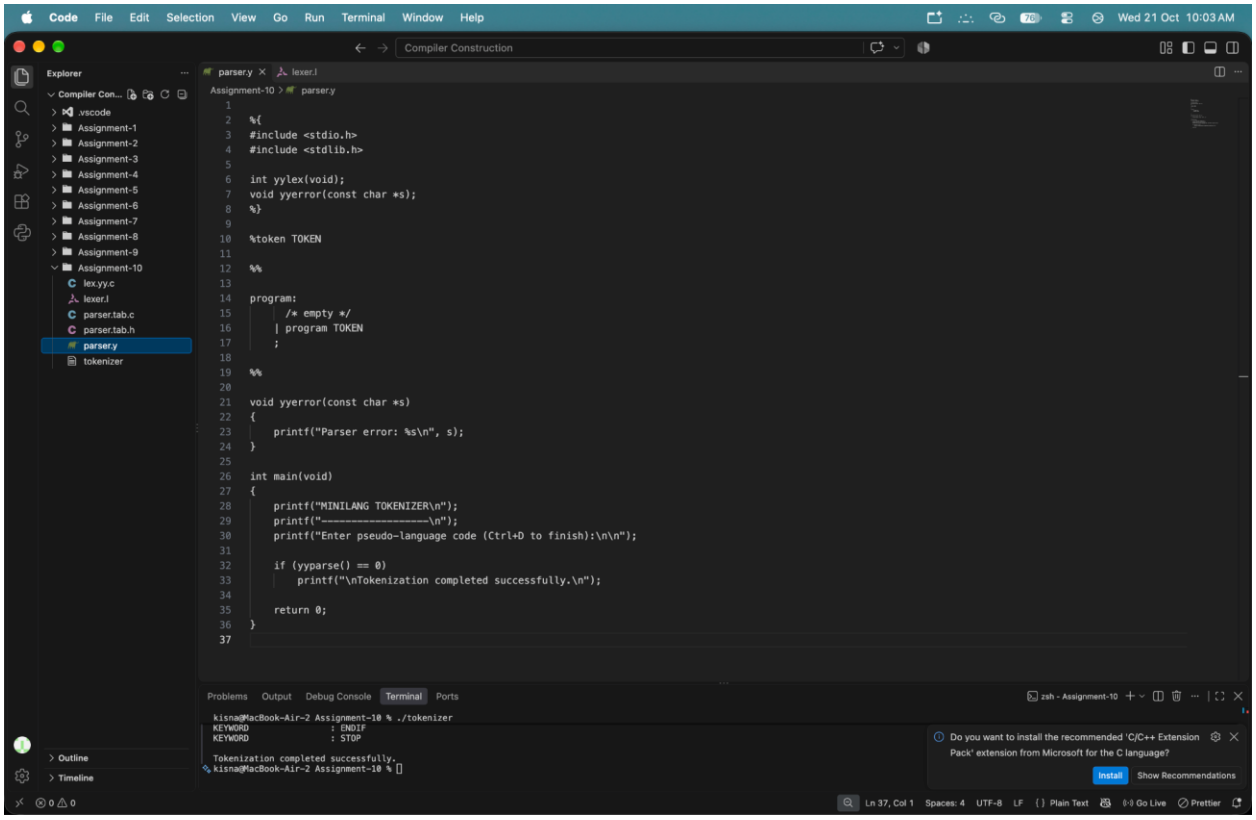
Theory

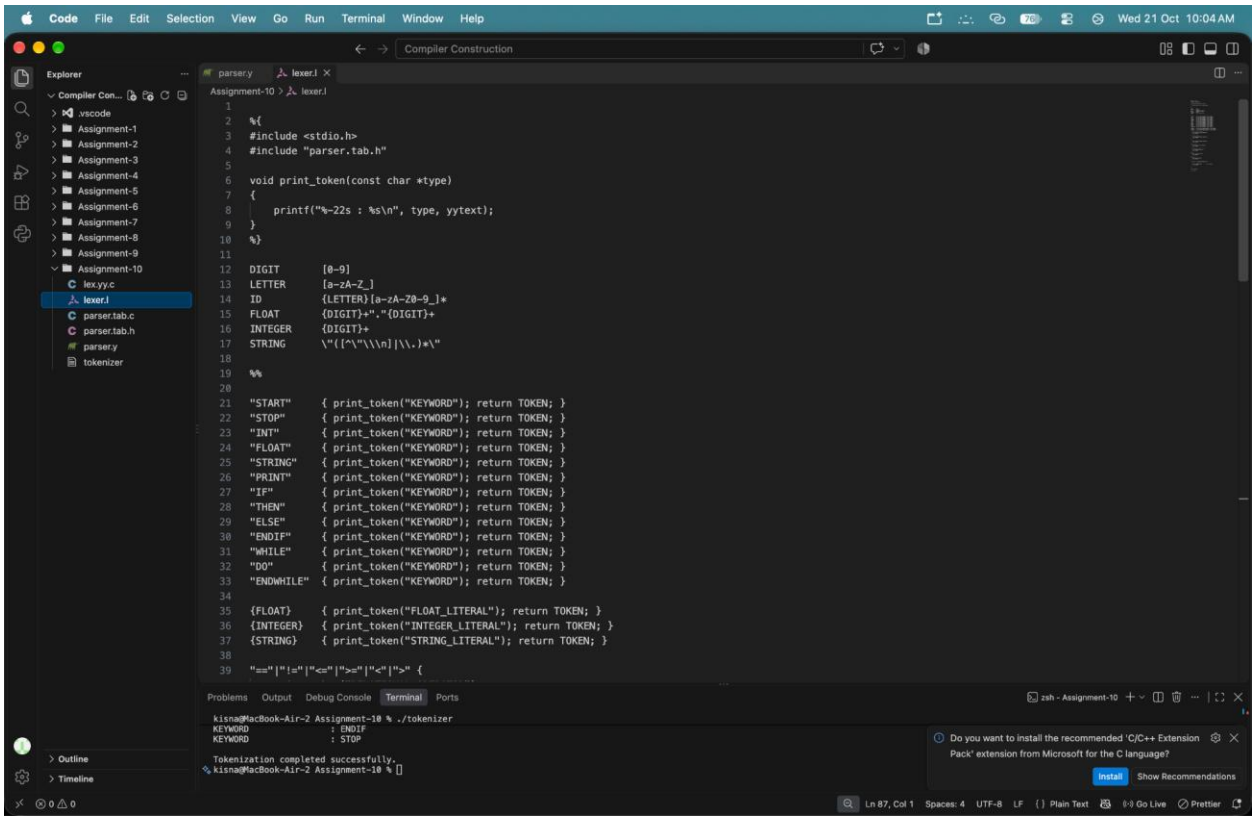
A tokenizer, also known as a lexical analyzer, is the first phase of a compiler. It reads the source code as a sequence of characters and groups them into meaningful units called tokens. Each token belongs to a specific category, such as a keyword, identifier, constant, operator, delimiter, or string literal.

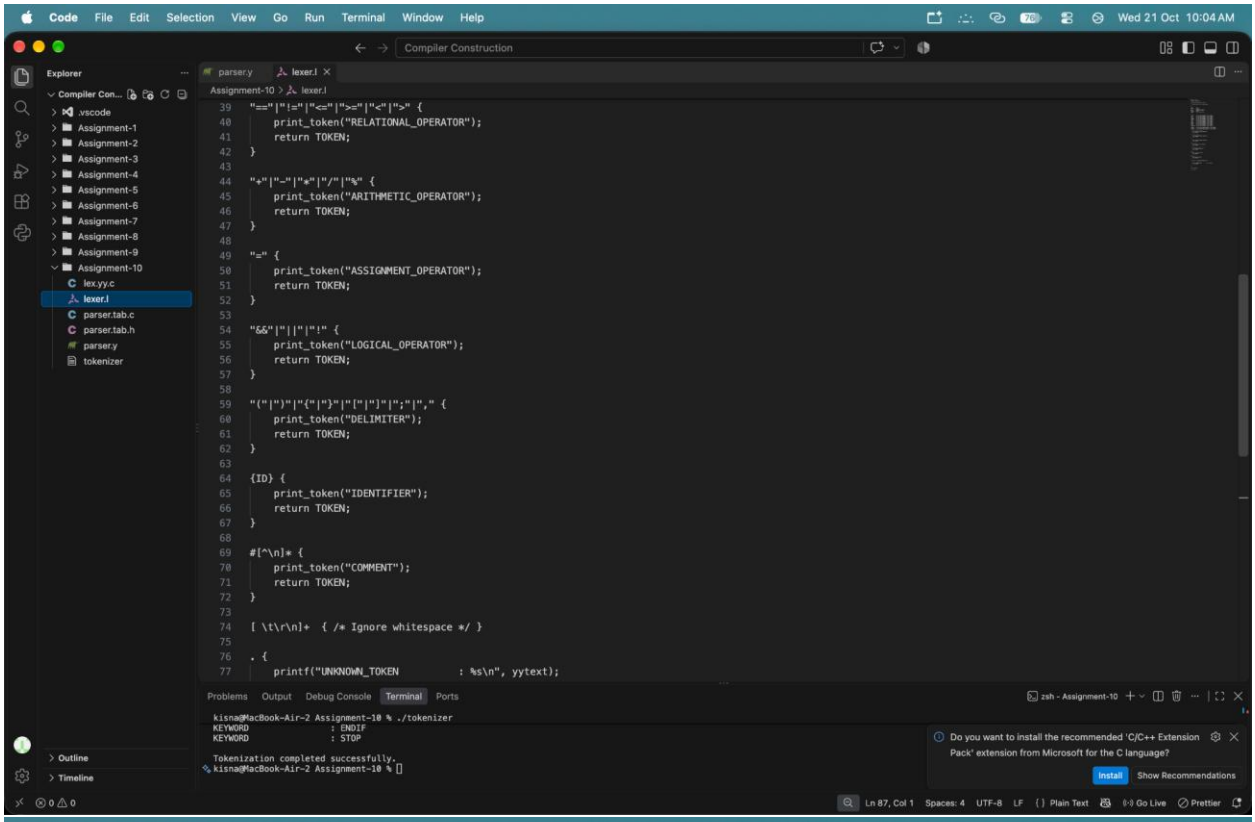
In this assignment, a mini tokenizer is designed for a custom pseudo programming language called MiniLang using Flex and Bison. The language supports keywords such as START, STOP, INT, FLOAT, PRINT, IF, and ELSE, along with identifiers, numeric literals, strings, arithmetic operators, relational operators, assignment operators, and delimiters.

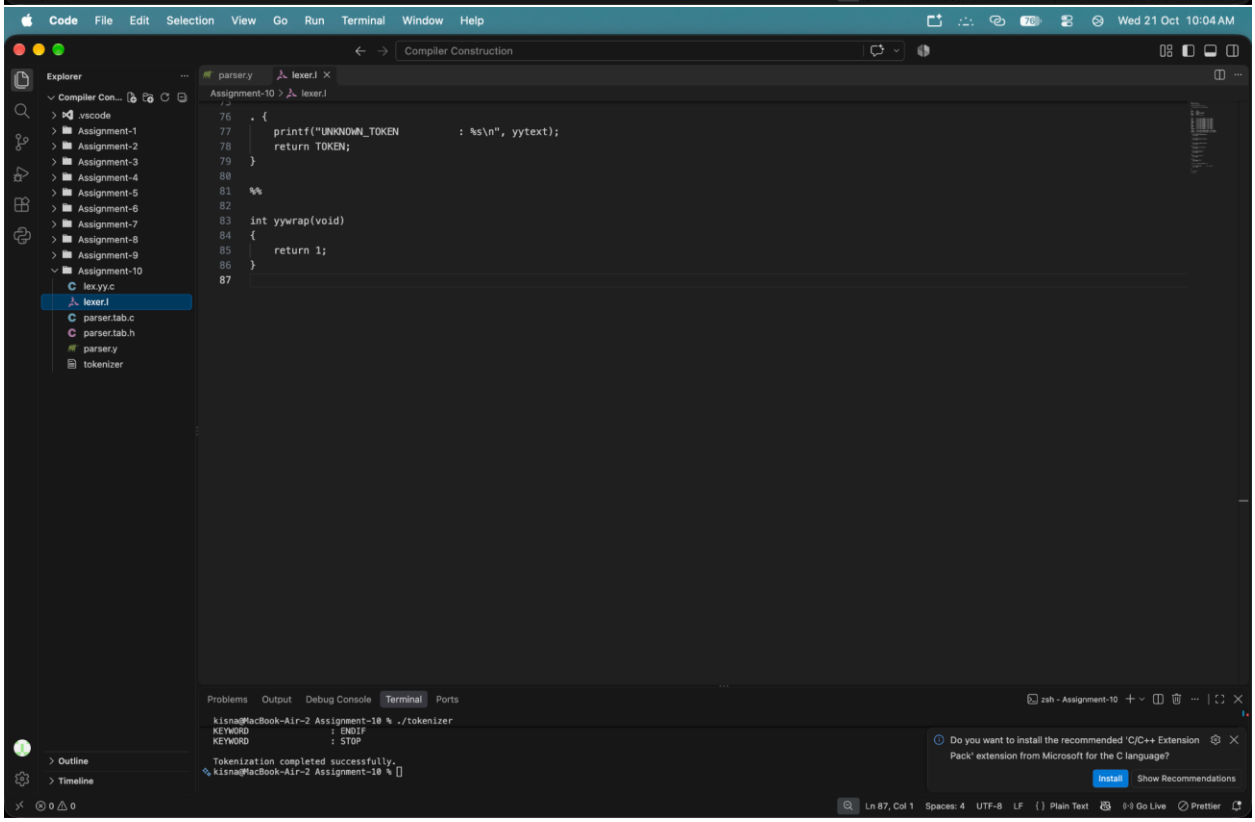
Regular Expressions are used to define token patterns. For example, [a-zA-Z_][a-zA-Z0-9_]* recognizes identifiers, [0-9]+ recognizes integer literals, and [0-9]+\.[0-9]+ recognizes floating-point literals. Specific patterns are defined for keywords, operators, strings, and delimiters to distinguish between different token categories.

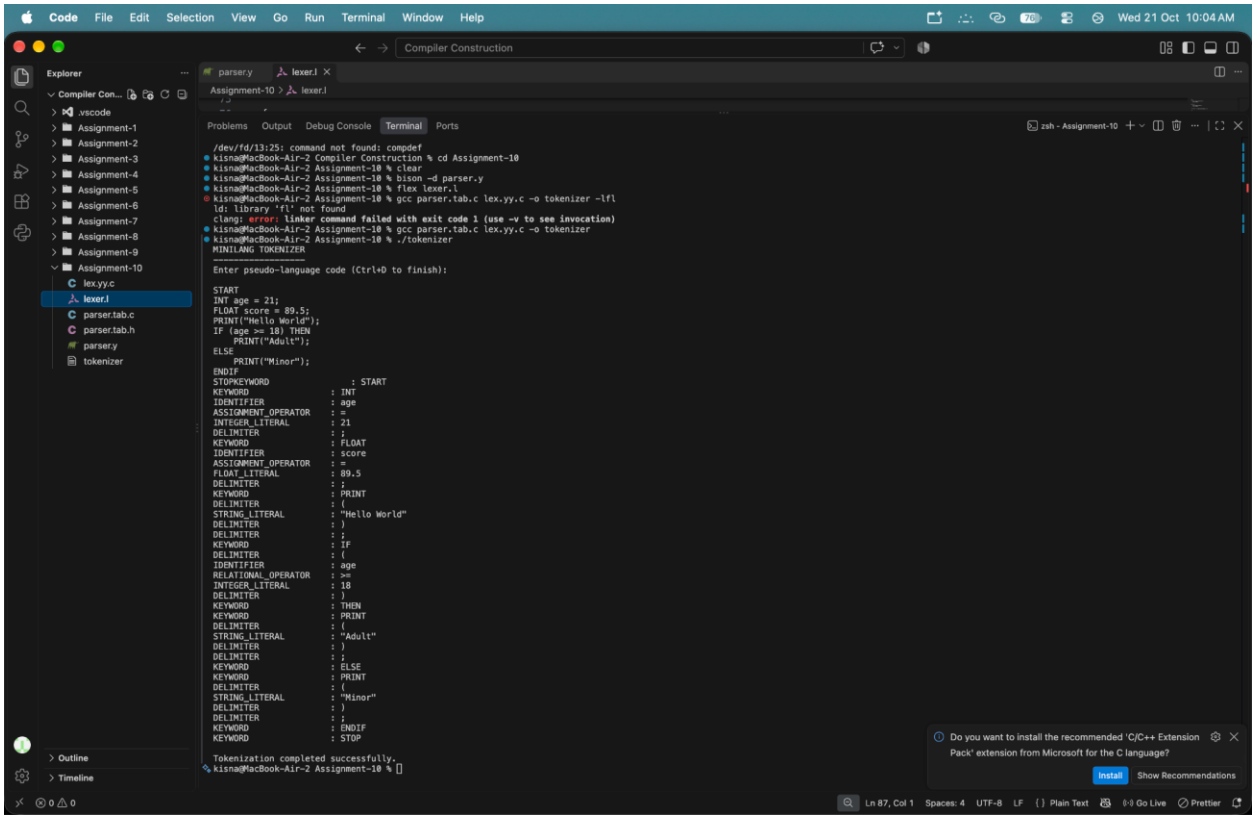
Screenshots and Execution Results






Code File Edit Selection View Go Run Terminal Window Help
Wed 21 Oct 10:04 AM
Compiler Construction
Explorer
Compiler Con...
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
Assignment-10
lex.yy.c
lexer.l
parser.tab.c
parser.tab.h
parser.y
tokenizer
parser.y
lexer.l
Assignment-10 > lexer.l
"=="|"!="|"<="|">="|"<"|">" {
print_token("RELATIONAL_OPERATOR");
return TOKEN;
}
"+"|"-"|"*"|"/"|"%" {
print_token("ARITHMETIC_OPERATOR");
return TOKEN;
}
"=" {
print_token("ASSIGNMENT_OPERATOR");
return TOKEN;
}
"&&"|"||"|"!" {
print_token("LOGICAL_OPERATOR");
return TOKEN;
}
"("|")"|"{"|"}"|"["|"]"|";"|"," {
print_token("DELIMITER");
return TOKEN;
}
{ID} {
print_token("IDENTIFIER");
return TOKEN;
}
#[^\n]* {
print_token("COMMENT");
return TOKEN;
}
[\t\r\n]+ { /* Ignore whitespace */ }
. {
printf("UNKNOWN_TOKEN : %s\n", yytext);
Problems Output Debug Console Terminal Ports
zsh - Assignment-10
kisna@MacBook-Air-2 Assignment-10 % ./tokenizer
KEYWORD : ENDIF
KEYWORD : STOP
Tokenization completed successfully.
kisna@MacBook-Air-2 Assignment-10 %
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 87, Col 1 Spaces: 4 UTF-8 LF Plain Text Go Live Prettier


Code File Edit Selection View Go Run Terminal Window Help
Wed 21 Oct 10:04 AM
Compiler Construction
Explorer
Compiler Con...
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
Assignment-10
lex.yy.c
lexer.l
parser.tab.c
parser.tab.h
parser.y
tokenizer
parser.y
lexer.l
Assignment-10 > lexer.l
. {
printf("UNKNOWN_TOKEN : %s\n", yytext);
return TOKEN;
}
%%
int yywrap(void)
{
return 1;
}
Problems Output Debug Console Terminal Ports
zsh - Assignment-10
kisna@MacBook-Air-2 Assignment-10 % ./tokenizer
KEYWORD : ENDIF
KEYWORD : STOP
Tokenization completed successfully.
kisna@MacBook-Air-2 Assignment-10 %
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 87, Col 1 Spaces: 4 UTF-8 LF Plain Text Go Live Prettier


Code File Edit Selection View Go Run Terminal Window Help
Wed 21 Oct 10:04 AM
Compiler Construction
Explorer
Compiler Con...
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
Assignment-10
lex.yy.c
lexer.l
parser.tab.c
parser.tab.h
parser.y
tokenizer
parser.y
lexer.l
Assignment-10 > lexer.l
Problems Output Debug Console Terminal Ports
zsh - Assignment-10
/dev/fd/13:25: command not found: compdef
kisna@MacBook-Air-2 Compiler Construction % cd Assignment-10
kisna@MacBook-Air-2 Assignment-10 % clear
kisna@MacBook-Air-2 Assignment-10 % bison -d parser.y
kisna@MacBook-Air-2 Assignment-10 % flex lexer.l
kisna@MacBook-Air-2 Assignment-10 % gcc parser.tab.c lex.yy.c -o tokenizer -lfl
ld: library 'fl' not found
clang: error: linker command failed with exit code 1 (use -v to see invocation)
kisna@MacBook-Air-2 Assignment-10 % gcc parser.tab.c lex.yy.c -o tokenizer
kisna@MacBook-Air-2 Assignment-10 % ./tokenizer
MINILANG TOKENIZER
Enter pseudo-language code (Ctrl+D to finish):
START
INT age = 21;
FLOAT score = 89.5;
PRINT("Hello World");
IF (age >= 18) THEN
PRINT("Adult");
ELSE
PRINT("Minor");
ENDIF
STOPKEYWORD : START
KEYWORD : INT
IDENTIFIER : age
ASSIGNMENT_OPERATOR : =
INTEGER_LITERAL : 21
DELIMITER : ;
KEYWORD : FLOAT
IDENTIFIER : score
ASSIGNMENT_OPERATOR : =
FLOAT_LITERAL : 89.5
DELIMITER : ;
KEYWORD : PRINT
DELIMITER : (
STRING_LITERAL : "Hello World"
DELIMITER :)
DELIMITER : ;
KEYWORD : IF
DELIMITER : (
IDENTIFIER : age
RELATIONAL_OPERATOR : >=
INTEGER_LITERAL : 18
DELIMITER :)
KEYWORD : THEN
KEYWORD : PRINT
DELIMITER : (
STRING_LITERAL : "Adult"
DELIMITER :)
DELIMITER : ;
KEYWORD : ELSE
KEYWORD : PRINT
DELIMITER : (
STRING_LITERAL : "Minor"
DELIMITER :)
DELIMITER : ;
KEYWORD : ENDIF
KEYWORD : STOP
Tokenization completed successfully.
kisna@MacBook-Air-2 Assignment-10 %
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 87, Col 1 Spaces: 4 UTF-8 LF Plain Text Go Live Prettier